Alexandra Postolaki, Takuya Paipoovong
posto022@umn.edu, paipo001@umn.edu
CSCI 5105 - 3/29/26
Professor Jon Weissman

Deliverable A: Accuracy vs. Vectors Searched as Repartition Size Changes:

- Smaller threshold usually means more partitions and fewer vectors searched, but more partitions may also hurt result quality if routing becomes less accurate.

Hypothesis: As the repartition threshold increases, we expect the search accuracy to improve while the number of vectors searched also increases. Larger thresholds result in fewer, larger nodes, which contain more data and therefore should increase the likelihood that relevant records are found during search. On the other hand, smaller thresholds would create more partitions which would reduce the number of vectors searched but will also increase the risk of missing important results that are more relevant. The decrease in accuracy with more partitions could most likely be the result of how data is partitioned and how the routing itself is performed. When a node splits, the data is separated into different partitions that create boundaries which may divide similar records across nodes. However, the centroids are only an average representation of the data within a node, so the routing decisions are an approximation that can lead to cases where the controller selects a node that is close to the query searched, but not the node that holds the most relevant data/records. Also, once the search is bubbled down to a single node, it doesn't check other, similar nodes for the possibility of greater results or results of higher-quality located there.

- Average number of vectors searched per question: (Cost)

Maximum Vector Per Node:	Avg. # of vectors searched per question
50	114.73
100	127.19
200	141.19
500	351.77
1000	833.49
2000	1269.26

- Average searched fraction, defined as vectors_searched / total_vectors_in_corpus:

Maximum Vector Per Node:	Avg. searched fraction
50	0.1096

100	**0.0963**
200	0.1069
500	0.2663
1000	0.6310
2000	0.9608

** Here, the average searched fraction between 50-200, 100 vectors per node shows a small dip. Possibly related to how partitioning evolves based on insertion order and the overall number of nodes. At small thresholds, the frequent splits create many small and noisy partitions. This leads to less stable routing decisions. As the threshold is increased to a max number of nodes being 100, nodes become slightly larger and centroids are more stable which results in more effective routing and a temporary reduction in search cost. However, as the threshold continues to increase, the number of nodes decreases and each node contains more vectors which would cause more vectors to be searched (the numerator) and, therefore, the search fraction to rise again. Because the partitioning depends on the incremental centroid updates and insertion order, these effects are not perfectly smooth as seen in the table above. This leads to small variations like the observerd dip mentioned.

- Average returned score for the top results:

Maximum Vector Per Node:	Avg. returned score for top results
50	0.5751
100	0.5674
200	0.5665
500	0.5857
1000	0.5876
2000	0.6013

** Something to note later, we went from 114.73 to 1269.25 for average number of vectors searched per question (a greater cost) as the maximum number of vectors per node increased, but the accuracy didn't improve by much (from 0.5751 to 0.6013). Is that tradeoff really worth it? It's like a huge cost increase for a droplet of accuracy increase.

- Average oracle score from the scored question file: (Ideal quality of answer)

Maximum Vector Per Node:	Avg. oracle score from scores question file

50	0.5812
100	0.5812
200	0.5812
500	0.5812
1000	0.5812
2000	0.5812

- A comparison/plot showing how search quality changes as the split threshold changes

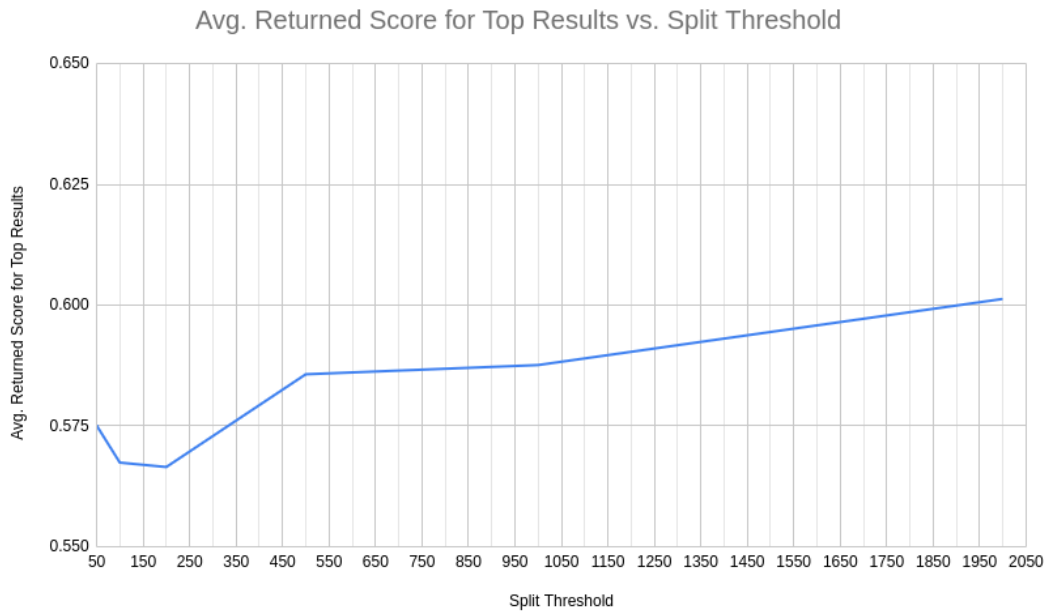

Figure 1. Average Returned Score for Top Results vs. Split Threshold (Maximum Number of Vectors Per Node).

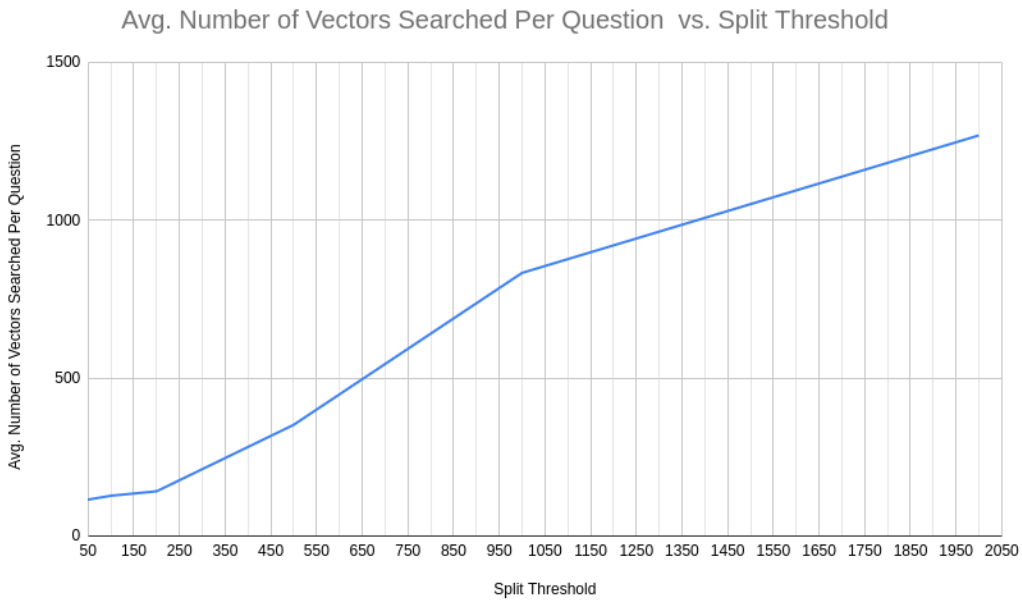

Figure 2. Average Number of Vectors Search Per Question vs. Split Threshold (Maximum Number of Vectors Per Node).

From the results shown in the tables and Figures 1 & 2, the hypothesis appears to be supported, though in a slightly unexpected way. As the repartition threshold increases, the trend of the number of vectors searched grows significantly. The unexpected part was that the search accuracy appeared to only improve slightly. It still, however, confirms that increasing node size improves result quality but at a more disproportionately higher cost computationally.

Deliverable B: Does Insertion Order Matter?

CODE USED TO GENERATE RESULTS FOR DELIVERABLE B FOUND IN ingestion.py . RUN AS INSTRUCTED IN PROJECT OUTLINE AND USE put_grouped_corpus.

Hypothesis: Insertion order affects efficiency and partition structure. Specifically, efficiency will be better for grouped ingestion as, according to the project outline and the idea behind centroids in general, centroids are updated incrementally. When similar data is inserted together early, centroids become more representative of coherent, nearby topics and leads to more accurate routing and partitions that are structured better.

- Does the final partition layout change? Yes. Using a maximum number of vectors per node to be 1,000 and comparing cluster_status.py outputs for shuffled and grouped ingestion, we see the following results:

Node:	Shuffled Full Corpus	Grouped Full Corpus
Node 0	- Num Vectors: [**276**] - Mean Score: [0.473918…]	- Num Vectors: [**531**] - Mean Score: [0.509711…]

	- Score Stdev: [0.105147…]	- Score Stdev: [0.112320…]
Node 1	- Num Vectors: [**997**] - Mean Score: [0.507044…] - Score Stdev: [0.104786…]	- Num Vectors: [**733**] - Mean Score: [0.489984…] - Score Stdev: [0.128421…]

In the shuffled case, the nodes are quite unbalanced (276 vs. 997). Whereas the grouped ingestion produces a ore balanced distribution (531 vs. 733). This indicates that insertion order affects how data is partitioned across nodes. The shuffled corpus introduces noise early which leads to less structures centroids and also uneven splitting. For grouped ingestion, however, the results point to cleaner centroids and more coherent partitions.

- Does the average number of vectors searched change?

```
======================================================================
NORMALIZED SCORING SUMMARY
======================================================================
Questions evaluated:      200
Total corpus vectors:     1321
Average hit rate:         0.5345
Average returned score:   0.5884
Average oracle score:     0.5812
Average score accuracy:   0.9781
Average vectors searched: 688.35
Average search fraction:  0.5211
FINAL TOTAL SCORE:        1.9735
======================================================================
Per-question metric:
  efficiency_score(q) = score_accuracy(q) / search_fraction(q)
Final total score:
  average efficiency_score(q) across all questions
======================================================================
```

Yes. For a maximum of 1,000 vectors per node, shuffled ingestion had an average of 833.49 vectors searched while a grouped ingestion had an average of 688.35. This represents a significant drop in search cost, pointing to the indication that grouped insertion leads to more efficient routing and partitioning.

- Does the returned search quality change? For shuffled ingestion, we see a returned search quality change of 0.5876 and for grouped ingestion a returned search quality of 0.5884. A higher average returned score for group ingestion than for shuffled ingestion. It's a slight improvement in quality from shuffled to grouped ingestion, but the difference is relatively small compared to the improvement in efficiency.
- Why do those differences happen in this system? These differences happen in this system because, as mentioned before, centroids are updated incrementally with early data having more influence on the centroid mean and routing depends on the centroids. The controller picks the nodes based on which centroid is closest, and when nodes split the data gets divided and new centroids are created which effects how future data is routed and assigned to nodes.

For grouped insertion, the splits appear more meaningful resulting in balanced nodes and clusters that make sense while shuffled insertions have a more random mix in all the data, the centroids are more messy, routing appears less accurate and splits have messy boundaries.

A similar example in machine learning: when feeding the computer pictures in a shuffled order, such as [‘dog’, ‘cat’, ‘dog’, ‘cat’, ‘dog’, ‘cat’] before a split occurs, the overall “mean” becomes more generalized. It’s messy and not quite specific if we’re looking to train a model to tell the difference between a dog or a cat. However, if similar data is presented together early before any split, such as [‘dog’, ‘dog’, ‘dog’, ‘dog’, ‘dog’, ‘cat’], the model can form more specific and accurate predictions leading to better organization and performance.

** Explanation should connect the observed behavior back to incremental centroid updates and split timing.

Deliverable C: Proposing a Better Repartitioning Scheme:

- The goal should be to minimize the number of vectors searched and preserve/improve the returned search quality.
- What metadata the controller and nodes would need:
- How routing would work:
- How repartitioning would work:
- Why you expect it to improve the accuracy versus search-cost tradeoff:
- What new complexity or overhead your approach introduces:

To introduce a better partitioning scheme we must iterate the issues and the considerations we are following before proceeding.

Issue #1:

The current issues align with having only searching into one node and its materials that lie within itself which can cut costs but at the expense of accuracy. Since relevant records may exist in other nodes that aren’t considered. This leads to further complications by the fact that nodes work on their own and have no awareness of data stored in other partitions and nodes.

Issue #2:

Following this further we determined that the matter of how partitions are constructed pose obstacles with global structure. Knowing that records are assigned incrementally based on existing centroids, early insertions determine how clusters are formed. Therefore leading to inconsistent partitioning and varying split outcomes depending on insertion order.

Issue #3:

Another major hurdle is that each record belongs to one node. Which provides no manner of sharing or duplicating records across nodes. As a result, records lying near cluster boundaries are forced into one single partition. Despite the fact that the record may be relevant to multiple clusters.

Issue #4:

The repartitioning scheme on its own is all determined by local events. When a node exceeds the capacity, it performs the k=2 split without considering the global structure. This leads to poorly balanced clusters which give way to inaccurate centroids and inadequate routing decisions. Moreover, there is no presence of a balancing or correcting partitions once they are created and in turn invites errors to persist over time.

Considering issues outlined above, we propose a system that considers how we search and partition the clusters. First, we look into improving search through evaluating the queries for similarities to all node centroids. Then, we select the top-3 closest centroids. The send to SearchLocal goes to the 3 groups to then merge results and return global top-k. We propose a top-3 system over the top-2 system to give more robustness in search queries for larger datasets (scalability). This also provides us overlap between 3 nodes if the vectors are split we can still obtain the information and address our boundary issues. Despite the system now searching a top-2 or top-1 the cost for something like this for top-k nodes the gain in accuracy would slowly decrease over time as we begin to search through the entire system versus a subset of centroids. Taking into consideration that cost can start to play a factor with top-3 closest centroids, we are convinced that combining that logic with a new repartitioning scheme the cost can be overlooked.

The leverage gained by searching through top-3 closest centroids, we are able to maintain our scheme of splitting with k=2. Improving on scheme we make a strict balance constraint that follows $|\text{cluster } 1| \approx |\text{cluster } 2|$. This is to avoid the uneven splits that occur like 90/10, 70/30 etc. Alongside this we introduce the idea of compactness in a cluster to minimize average distance to centroid. Then this gives way to separation between clusters so we can have better routing decisions so we would like to maximize the distance between centroids. In terms of the boundary of clusters we can duplicate a small part of data to prevent missing relevant results during splitting. This would require the controller to keep track of this as well.

- Metadata for Controller
    - Maintain node centroids & sizes
    - Cluster variance
        - A global measurement for clusters for overall report/logging
- Metadata for Storage Nodes
    - Local records
    - Centroid
    - Variance / cluster quality metrics

- Determine if split is needed to be evoked and report/logging

Limitations with proposed model:

The overhead introduced by our proposal is an increase in query processing cost. The queries are evaluated against three nodes instead of the one. This introduces a higher number of vectors searched and additional RPC calls. Through both of these we see higher levels of computational and network overhead in the possibility of increased latency due to waiting for responses from the three nodes. However, these tradeoffs are balanced with capped overhead costs and maintaining low full search through the system. The controller on the other hand receives a small overhead in merging and reranking operations on the results but proven negligible compared to the vector similarity operation. Most importantly, we have no additional storage overhead when insertions remain single-node. Therefore the increased cost is moderate and controlled, while enhanced search accuracy conditions justify the provided tradeoff.

Deliverable D: Extending the Evaluation Beyond the Sample:

- Extend evaluation so it runs across all questions in question_set/questions_scored.jsonl:
- Report aggregate metrics for the full question set:

Changes have been made as needed. We evaluated the system on the full question set and noticed consistent results with earlier runs. The system maintained high score accuracy with moderate search cost for a maximum number of vectors per node of 1,000, confirming the tradeoff between efficiency and accuracy. These results demonstrate taht our findings generalize to the full dataset.

```
================================================================================
NORMALIZED SCORING SUMMARY
================================================================================
Questions evaluated:      200
Total corpus vectors:     1321
Average hit rate:         0.5307
Average returned score:   0.5862
Average oracle score:     0.5812
Average score accuracy:   0.9788
Average vectors searched: 721.59
Average search fraction:  0.5462
FINAL TOTAL SCORE:        2.0179
================================================================================
Per-question metric:
  efficiency_score(q) = score_accuracy(q) / search_fraction(q)
Final total score:
  average efficiency_score(q) across all questions
================================================================================
```

Questions to Consider discussing:

- When did smaller partitions help?

- When do smaller partitions hurt?
- Under what conditions did insertion order have the biggest effect?
- Is one-node search too restrictive for some queries?
- Would searching two nodes instead of one be a good tradeoff?
- Does your alternative scheme improve quality, cost, or both?

→ Wrap up with the goal (study how a distributed semantic store behaves when you trade off accuracy, efficiency, online scalability, and simplicity of implementation).

This project demonstrates the tradeoff between efficiency and accuracy in a distributed semantic store and search system. The system also shows online scalability as it scales with more data added in by splitting nodes over time without needing to restart or reorganize everything. Smaller partitions improved efficiency by reducing the number of vectors searched, but hurt accuracy when relevant data was split across node and missed due to the single-node search. Insertion order had a significant impact on partition structure, as grouped ingestion produced more balanced nodes and more efficient routing compared to shuffled data, due to the incremental nature of centroid updates. These results highlight taht one-node search can be too restrictive for queries near partition boundaries. It suggests that searching multiple nodes would improve recall at a moderate increase of cost. Our proposed approach of querying multiple nodes and enforcing more balanced partitions improves robustness and accuracy while maintaining reasonable efficiency. It does, however, introduce additional computational overhead and system complexity. Overall, this project illustrates how design choices in partitioning, routing, and query strategy affect the balance between accuracy, efficiency, online scalability, and simplicity of implementation in distributed systems.